

Contents

Taller 3 - Programación discreta · Documento de explicación	1
Bloque A · Criptografía	1
Punto 1 · Cifrado César	1
Punto 2 · RSA de juguete	2
Punto 3 · MPC básico: promedio sin mostrar los datos	3
Bloque B · Grafos	4
Punto 4 · Ruta más corta (Dijkstra)	4
Punto 5 · Cierre de una estación	5
Punto 6 · Coloreo de grafos (exámenes sin choques)	6
Bloque C · Álgebra de Boole, Shannon y computación cuántica	8
Punto 7 · Tablas de verdad y circuitos lógicos	8
Punto 8 · Simplificación booleana (Quine-McCluskey)	9
Punto 9 · Entropía de Shannon	10
Punto 10 · Primer simulador cuántico	11
Cierre	13

Taller 3 - Programación discreta · Documento de explicación

Matemáticas Discretas I — Universidad Nacional de Colombia Integrante: Julián Andrés Foglia Wilches · Lenguaje: Python 3.10+ · Sin librerías externas

Para cada punto se responde: **(1)** qué problema resuelve el programa, **(2)** qué idea matemática usa, **(3)** cómo se ejecuta, **(4)** qué pruebas se hicieron y **(5)** qué limitaciones tiene, además de la pregunta específica que pide el enunciado en «Para la documentación».

Todos los fragmentos de salida que aparecen abajo son copias literales de una ejecución real de los programas. Las demostraciones usan semillas fijas y por eso son reproducibles, con dos excepciones deliberadas: los totales de los servidores del punto 3 (que se sortean con System-Random y cambian en cada corrida, que es justamente lo que los hace seguros) y las frecuencias observadas en las mediciones del punto 10, que varían dentro del margen estadístico.

Bloque A · Criptografía

Punto 1 · Cifrado César

1. Problema. Cifrar y descifrar un texto con un desplazamiento fijo k , y romper el cifrado cuando k es desconocido.

2. Idea matemática. Se numeran las letras $A \rightarrow 0, \dots, Z \rightarrow 25$ y se trabaja en el grupo aditivo $\mathbb{Z}_{26}$:

cifrar: $E_k(x) = (x + k) \bmod 26$

descifrar: $D_k(y) = (y - k) \bmod 26$

E_k es una biyección de $\mathbb{Z}_{26}$ en sí mismo (sumar una constante en un grupo siempre lo es), y su inversa es sumar $-k$. Los caracteres que no son letras (espacios, signos, dígitos) se dejan intactos, y se recuerda si la letra era mayúscula o minúscula para reponerla igual.

Para la documentación — ¿por qué descifrar usa el desplazamiento contrario? Porque D_k es literalmente la función inversa de E_k : $D_k(E_k(x)) = (x + k - k) \bmod 26 = x$. Cifrar «corre» las letras hacia adelante en el ciclo de 26 posiciones, así que la única forma de volver

al punto de partida es correrlas hacia atrás la misma cantidad. En el programa, `descifrar(t, k)` es exactamente `cifrar(t, -k)`, y el módulo se encarga del borde: `cifrar("ABC", -1) == "ZAB"`.

Para la documentación — ¿por qué el ataque de fuerza bruta es posible? Porque el **espacio de claves tiene 26 elementos** (25 útiles: $k = 0$ no cifra). La seguridad de un cifrado depende de que probar todas las claves sea imposible; aquí probar todas es instantáneo. Además, cada k produce un texto legible o ilegible, así que el atacante distingue el correcto sin conocer la clave. El programa aprovecha eso: calcula un χ^2 comparando la frecuencia de letras de cada candidato con las frecuencias del español y ordena los 26 resultados. La clave correcta suele quedar primera con un χ^2 mucho menor:

```
Mensaje interceptado: WL LCTEXPETNL XZOFWLC SLNP CPGPCDTMWP PW NTQCLOZ
k=11 chi= 18.6 LA ARITMETICA MODULAR HACE REVERSIBLE EL CIFRADO
k=24 chi= 127.4 YN NEVGZRGVPN ZBQHYNE UNPR ERIREFVOYR RY PVSENQB
```

3. Ejecución. `python3 src/cripto/cesar.py` (o `python3 run_all.py 1`).

4. Pruebas. Caso obligatorio (HOLA UNAL, $k=3 \rightarrow$ KR0D XQD0); ida y vuelta para k de -30 a 30 en varios textos; conservación exacta de espacios, signos, dígitos y mayúsculas; el «wrap-around» ($XYZ + 3 = ABC$); que la fuerza bruta liste las 26 opciones y contenga el texto correcto; y que `romper()` recupere la clave exacta para $k \in \{3, 7, 11, 19, 25\}$ en un texto largo.

5. Limitaciones. El alfabeto es de 26 letras sin Ñ ni tildes (las tildes pasan sin cifrar, lo que filtra información). El ranking del ataque supone texto en español y **necesita textos largos**: en mensajes de dos o tres palabras el χ^2 puede señalar mal, aunque el texto correcto sigue estando entre los 26 candidatos que el programa imprime.

Punto 2 · RSA de juguete

1. Problema. Generar el par de llaves de RSA a partir de dos primos p , q y un exponente público e , cifrar un número y descifrarlo.

2. Idea matemática.

Paso	Fórmula
Módulo	$n = p \cdot q$
Función de Euler	$\phi(n) = (p-1)(q-1)$
Llave privada	$d \equiv e^{-1} \pmod{\phi(n)}$
Cifrado	$C \equiv M^e \pmod{n}$
Descifrado	$M \equiv C^d \pmod{n}$

Funciona porque $e \cdot d = 1 + t \cdot \phi(n)$ para algún entero t , y entonces

$$C^d \equiv M^{(e \cdot d)} \equiv M^{(1 + t \cdot \phi(n))} \equiv M \cdot (M^{\phi(n)})^t \equiv M \pmod{n}$$

por el teorema de Euler. Se implementaron a mano tres piezas:

- **Euclides extendido** (iterativo): devuelve (g, x, y) con $a \cdot x + b \cdot y = g = \gcd(a, b)$.
- **Inverso modular**: de Bézout, si $e \cdot x + \phi(n) \cdot y = 1$ entonces $d = x \bmod \phi(n)$.
- **Exponenciación modular** por cuadrados sucesivos: recorre los bits del exponente, hace $O(\log e)$ multiplicaciones y nunca construye el número gigante M^e .

Para la documentación — el papel de los primos, del inverso modular y de la congruencia.

- Los **primos** sirven para dos cosas. Primero, permiten calcular $\phi(n)$ fácilmente: si $n = p \cdot q$ con p, q primos distintos, entonces $\phi(n) = (p-1)(q-1)$. Quien no conoce la factorización de n no puede calcular $\phi(n)$ y por lo tanto no puede obtener d . Segundo, la seguridad real de RSA descansa en que factorizar n en primos grandes es computacionalmente costoso.
- El **inverso modular** es la puerta trasera legítima: d deshace lo que hace e porque $e \cdot d \equiv 1 \pmod{\phi(n)}$. Existe si y solo si $\gcd(e, \phi(n)) = 1$; el programa comprueba esa condición y avisa si falla:
 e no coprimo con ϕ : detectado -> $e = 13$ no es valido:
 $\gcd(e, \phi(n)) = 13 \neq 1$, entonces e no tiene inverso modulo $\phi(n) = 3120$
- La **congruencia** módulo n es lo que hace que todo esto sea aritmética finita: los mensajes viven en $\{0, \dots, n-1\}$ y elevar a potencias nunca sale de ese conjunto. Cifrar es una permutación de ese conjunto finito y d es la permutación inversa.

Caso obligatorio del taller (salida literal del programa):

```
n      = 3233
phi    = 3120
e      = 17
d      = 2753
C = 65^17 mod 3233 = 2790
M = C^2753 mod 3233 = 65
```

3. Ejecución. `python3 src/cripto/rsa_juguete.py` (o `python3 run_all.py 2`).

4. Pruebas. El caso obligatorio completo; identidad de Bézout en Euclides extendido para varios pares; $a \cdot a^{-1} \equiv 1$ en varios módulos; comparación de la exponenciación modular propia contra `pow()` de Python; ida y vuelta para **los 187 mensajes posibles** con $p=17, q=11, e=7$; un texto completo carácter por carácter; y cuatro validaciones de error (e no coprimo, p no primo, $p = q$, mensaje fuera de rango).

5. Limitaciones. Es un RSA **de juguete**: $n = 3233$ se factoriza a mano. No hay relleno (padding), y al cifrar carácter por carácter la misma letra produce siempre el mismo bloque, así que el texto cifrado es vulnerable a análisis de frecuencias — exactamente el problema del punto 1. Tampoco se generan primos aleatoriamente ni se comprueba primalidad de forma eficiente (se usa división de prueba, suficiente para primos pequeños).

Punto 3 · MPC básico: promedio sin mostrar los datos

1. Problema. Varios estudiantes quieren conocer la **suma y el promedio** de sus notas sin que nadie (ni los servidores) vea una nota individual.

2. Idea matemática (reparto aditivo módulo M). Cada nota x se parte en tres sumandos:

```
s1, s2  uniformes al azar en  $\mathbb{Z}_M$ 
s3 = (x - s1 - s2) mod M
==>    x  $\equiv$  s1 + s2 + s3 (mod M)
```

El servidor i recibe solo s_i de cada nota y publica **un único número**: $T_i = \sum (\text{partes que recibió}) \pmod M$. Como la suma es asociativa y conmutativa, sumar «por columnas» (servidores) o «por filas» (estudiantes) da lo mismo:

$(T_1 + T_2 + T_3) \pmod M = (\sum \text{notas}) \pmod M$

y como $\sum \text{notas} \leq 50 \cdot n < M = 1000003$, el resultado modular **es** la suma real. El promedio es esa suma dividida por n .

Para la documentación — ejemplo pequeño de que ninguna parte revela x. Tomemos $x = 40$ y $M = 1000003$. Una ejecución real dio:

La nota 40 se partió en: [339563, 993908, 666575]
Solo la suma de las tres da la nota: 40

Efectivamente $339563 + 993908 + 666575 = 2000046 = 2 \cdot 1000003 + 40$. Ahora, ¿qué sabe el servidor 1 al ver $s_1 = 339563$? **Nada:** s_1 se sorteó uniformemente y de manera independiente de x , así que ese mismo valor era igual de probable para cualquier nota entre 0 y 50. Peor aún, ni siquiera dos servidores coaligados aprenden algo: dadas s_1 y s_2 , **para cada** nota hipotética x' existe una tercera parte $s_3' = (x' - s_1 - s_2) \bmod M$ que la explica perfectamente. Es el mismo argumento del cuaderno de un solo uso (*one-time pad*): la máscara aleatoria borra toda la información. Solo la suma de las tres partes reconstruye x . La prueba `test_ninguna_parte_sola_revela_la_nota` verifica justamente eso: recorre las 51 notas posibles y muestra que todas son compatibles con las dos primeras partes.

Ejemplo mínimo del taller (salida literal):

```
Salida publica del protocolo -> suma = 150, promedio = 37.5
Totales que publico cada servidor (solo estos numeros salen):
S1: 548744
S2: 268382
S3: 183027
(548744 + 268382 + 183027) mod 1000003 = 150
```

3. Ejecución. `python3 src/cripto/mpc_promedio.py` (o `python3 run_all.py 3`).

4. Pruebas. Ejemplo mínimo (150 y 37.5); reconstrucción correcta para las 51 notas posibles; que ningún servidor tenga la lista original; listas de tamaño 1, 2, 7, 100 y 500 con suma y promedio correctos; validaciones de notas fuera del rango 0-50, no enteras y lista vacía.

5. Limitaciones. Es una **simulación en un solo proceso**: los tres «servidores» son objetos de Python, no máquinas separadas, y el reparto lo hace el mismo programa (en un protocolo real cada estudiante generaría sus partes localmente y las enviaría por canales cifrados). El modelo de seguridad es *honesto pero curioso* y con **no colusión total**: si los tres servidores se juntan, reconstruyen cada nota. Tampoco hay autenticación, así que un servidor podría mentir en su total y falsear la suma sin que nadie lo detecte.

Bloque B · Grafos

Punto 4 · Ruta más corta (Dijkstra)

1. Problema. Dada una red de transporte con tiempos de viaje, encontrar la ruta de menor tiempo total entre dos puntos y reportar la distancia y el camino.

2. Idea matemática. Se mantiene una estimación $\text{dist}[v]$ de la distancia desde el origen y un conjunto de vértices *cerrados* (con distancia definitiva). En cada paso se cierra el vértice pendiente con estimación mínima y se **relajan** sus aristas:

```
si  $\text{dist}[u] + \text{peso}(u,v) < \text{dist}[v]$ :
     $\text{dist}[v] = \text{dist}[u] + \text{peso}(u,v)$ 
     $\text{padre}[v] = u$ 
```

Los `padre[]` permiten reconstruir la ruta al final, caminando desde el destino hacia atrás. La cola de prioridad (`heapq`) es solo la estructura de datos que entrega el mínimo; la lógica del algoritmo está escrita en el módulo.

Para la documentación — ¿por qué Dijkstra necesita pesos no negativos? El invariante clave es: *cuando se extrae el vértice u con la menor estimación pendiente, esa estimación ya es óptima*. La justificación es que cualquier otro camino a u tendría que salir del conjunto cerrado por algún vértice pendiente w con $\text{dist}[w] \geq \text{dist}[u]$, y desde w seguir sumando pesos. **Si los pesos son ≥ 0 , sumar solo puede empeorar**, así que ese camino alternativo no puede ser más corto y $\text{dist}[u]$ es definitiva. Con un peso negativo el argumento se rompe: un tramo negativo posterior podría bajar el total, y el vértice ya cerrado quedaría con una distancia equivocada, sin que el algoritmo vuelva a revisarlo. Por eso el programa **rechaza pesos negativos al construir el grafo**:

Peso negativo rechazado -> peso negativo no permitido: A-B = -3

(Para pesos negativos existe Bellman-Ford, que relaja todas las aristas $|V|-1$ veces; se usa en las pruebas como algoritmo de contraste.)

Para la documentación — ¿qué significa que un camino sea óptimo? Que **ningún otro camino** entre los mismos extremos tiene peso total menor. Dos consecuencias que el programa comprueba: el peso de la ruta reportada coincide exactamente con la distancia calculada, y se cumple la **desigualdad triangular** $d(a, c) \leq d(a, b) + d(b, c)$ (si no se cumpliera, pasar por b daría un camino mejor que el «óptimo»). También vale la *subestructura óptima*: todo tramo de un camino óptimo es a su vez óptimo entre sus extremos — que es justo lo que permite guardar un solo padre[v] por vértice.

Grafo de prueba (data/ciudad.txt, 10 vértices y 15 aristas, cumple el mínimo de 8 y 12 que pide el enunciado):

Portal, Calle26, Museo, Centro, Universidad, Parque, Terminal, Hospital, Estadio, Vereda
Salida de ejemplo:

```
Portal -> Estadio      distancia =   12   ruta: Portal -> Terminal -> Estadio
Portal -> Parque       distancia =   14   ruta: Portal -> Calle26 -> Museo -
> Universidad -> Parque
Vereda -> Universidad distancia =   21   ruta: Vereda -> Terminal -> Estadio -
> Hospital -> Parque -> Universidad
Componentes separadas: A -> Y distancia = inf, ruta = (sin camino)
```

3. Ejecución. python3 src/grafos/dijkstra.py (o python3 run_all.py 4). El grafo se carga del archivo data/ciudad.txt; también se puede construir en código con GrafoPonderado.desde_lista([...]).

4. Pruebas. Grafo de control verificable a mano (el camino por B de peso 2 gana a la arista directa de peso 5); seis distancias conocidas de la ciudad; distancia 0 de un vértice a sí mismo; coherencia entre ruta y distancia para **todos** los pares; simetría en el grafo no dirigido; desigualdad triangular en todas las ternas; inf y ruta vacía cuando no hay camino; y comparación contra **Bellman-Ford** en 15 grafos aleatorios.

5. Limitaciones. Solo pesos no negativos. Devuelve **una** ruta óptima (si hay empates, no las enumera). Los pesos son estáticos: no modela horarios, trasbordos, capacidad ni congestión. Es la versión con montículo binario, $O((V+E) \log V)$, suficiente para grafos de este tamaño pero no optimizada para redes de millones de nodos.

Punto 5 · Cierre de una estación

1. Problema. Medir el impacto de cerrar una estación (vértice) o un tramo (arista): qué pares origen-destino se alargan, cuáles no cambian y cuáles quedan sin camino.

2. Idea matemática. Se calculan las rutas mínimas antes del cierre, se elimina el vértice/arista **sobre una copia** del grafo y se recalculan. Como quitar aristas solo puede **reducir** el conjunto de caminos disponibles, y el mínimo sobre un subconjunto es mayor o igual que el mínimo sobre el conjunto completo, se cumple $d_{\text{después}} \geq d_{\text{antes}}$ siempre. Por eso la columna «diferencia» nunca es negativa, y solo hay tres estados posibles: IGUAL, AUMENTA o DESCONECTADO (más ESTACIÓN CERRADA cuando el vértice eliminado es el propio origen o destino del par).

Para la documentación — qué se cerró y por qué produce (o no) impacto. Se probaron cuatro escenarios sobre los mismos 6 pares:

Escenario	Qué se cierra	Resultado
1	Vértice Centro	2 pares aumentan, 4 iguales
2	Vértice Terminal	1 par aumenta, 1 desconectado
3	Arista Museo–Centro	2 pares aumentan
4	Vértice Parque	0 aumentan (impacto casi nulo)

- **Centro sí importa** porque es un vértice de paso de grado alto y con aristas baratas (Museo–Centro = 2, Centro–Hospital = 4): varias rutas óptimas lo atravesaban. Al cerrarlo, Museo → Estadio pasa de 8 a 12 (+4) porque debe rodear por Universidad → Parque → Hospital, y Portal → Hospital pasa de 13 a 14. Aun así **nada se desconecta**: la red tiene ciclos alternativos.

Museo -> Estadio

antes : Museo -> Centro -> Hospital -> Estadio (8)

despues: Museo -> Universidad -> Parque -> Hospital -> Estadio (12)

- **Terminal es un punto de articulación.** Vereda cuelga únicamente de Terminal, así que al cerrarlo Vereda se aísla y el par Vereda → Museo pasa de 17 a inf. Ese es exactamente el caso que el programa debe detectar:

Vereda Museo 17 inf inf DESCONECTADO

- **Parque casi no importa:** ninguna ruta óptima entre los otros pares pasaba por él, así que todas las distancias quedan iguales; solo desaparece el par que tenía a Parque como destino.

La conclusión es la lectura estructural: el impacto de un cierre no depende del grado del vértice, sino de **cuántas rutas óptimas lo usan** y de si es un punto de articulación del grafo.

3. Ejecución. python3 src/grafos/cierre.py (o python3 run_all.py 5).

4. Pruebas. Seis pares por escenario (el enunciado pide ≥ 5); que la diferencia sea ≥ 0 cerrando **cada uno** de los 10 vértices; detección de desconexión con inf y ruta vacía; detección de aumento con valores exactos (13 → 14 al cerrar Centro); cierre de arista sin modificar el grafo original; cierre sin impacto; y error si no se indica exactamente un cierre.

5. Limitaciones. Se cierra un solo elemento por escenario (no combinaciones). Se mide el impacto en distancia, no en pasajeros, capacidad, ni en el redireccionamiento del flujo. Recalcula Dijkstra desde cero por cada consulta, lo cual es cómodo y claro pero no eficiente para redes grandes.

Punto 6 · Coloreo de grafos (exámenes sin choques)

1. Problema. Asignar franjas horarias a exámenes de modo que dos cursos con estudiantes en común nunca coincidan. Vértices = cursos, aristas = conflictos, colores = franjas.

2. Idea matemática. Algoritmo **voraz**: se recorren los vértices en algún orden y a cada uno se le asigna el color de menor índice que no usen sus vecinos ya coloreados (el «mex» del conjunto de colores prohibidos). Se usa el orden de **Welsh-Powell** (grado decreciente), que suele dar buenos resultados porque atiende primero a los vértices más restringidos.

Para la documentación — ¿por qué el voraz no garantiza el mínimo, pero sí una asignación válida?

- **Es válido siempre, por construcción:** el algoritmo *nunca elige* un color que ya tenga un vecino, así que al terminar no puede existir una arista monocromática. Además usa a lo sumo $\Delta + 1$ colores (Δ = grado máximo), porque un vértice de grado d tiene a lo más d colores bloqueados y siempre queda uno libre entre $d + 1$ opciones. En el grafo del taller: 3 colores usados, con cota $\Delta + 1 = 7$.
- **No es óptimo porque depende del orden** y decide sin mirar el futuro: un color asignado no se revisa después. El número cromático $\chi(G)$ es el mínimo real y calcularlo es NP-difícil; el voraz solo entrega una **cota superior**. El ejemplo canónico es el grafo *corona* (a_i unido a b_j solo si $i \neq j$), que es bipartito y por lo tanto necesita 2 colores, pero con un orden alternado el voraz gasta 3:

```
orden por lados : 2 colores -> {0: ['a1','a2','a3'], 1: ['b1','b2','b3']}
orden alternado : 3 colores -> {0: ['a1','b1'], 1: ['a2','b2'], 2: ['a3','b3']}
```

Los dos son válidos; uno simplemente desperdicia una franja. Lo mismo se ve en el grafo de cursos, donde según el orden salen 3 o 4 franjas:

```
orden alfabetico   -> 3 colores, valido: True
orden inverso      -> 4 colores, valido: True
```

Resultado sobre el grafo de conflictos (12 cursos, 24 conflictos — el enunciado pide ≥ 10 vértices):

```
Franja 1: Estadistica, Fisica, Logica, Programacion
Franja 2: BasesDatos, Calculo, Discretas, Filosofia, Quimica, Redes
Franja 3: Algebra, Historia, SistemasOp
Coloreo valido (ningun par adyacente comparte franja): True
Cota inferior: ['Algebra','Calculo','Programacion'] forman un triangulo -
> >= 3 franjas
```

Como existe un triángulo (Álgebra–Cálculo–Programación son mutuamente conflictivos), se necesitan al menos 3 franjas. El voraz encontró 3 $\implies$ **en este caso el resultado sí es óptimo**, y podemos demostrarlo con la cota inferior.

3. Ejecución. `python3 src/grafos/coloreo.py` (o `python3 run_all.py 6`).

4. Pruebas. Grafo con ≥ 10 vértices; coloreo válido y sin conflictos restantes; todos los vértices reciben color exactamente una vez; nunca se supera $\Delta + 1$; validez para **50 órdenes aleatorios**; grafos con número cromático conocido ($K_4 \rightarrow 4$, ciclo par $C_6 \rightarrow 2$, ciclo impar $C_5 \rightarrow 3$); el contraejemplo del grafo corona (2 vs 3 colores); y errores con órdenes incompletos o lazos.

5. Limitaciones. No calcula $\chi(G)$ exacto (sería exponencial). Solo modela conflictos binarios: no considera cuántos estudiantes comparten los cursos, ni capacidad de salones, ni preferencias de horario, ni que un estudiante no debería tener dos exámenes seguidos. La cota inferior implementada busca triángulos, no camarillas de tamaño arbitrario.

Bloque C · Álgebra de Boole, Shannon y computación cuántica

Punto 7 · Tablas de verdad y circuitos lógicos

1. Problema. Generar la tabla de verdad de una expresión booleana con variables A, B, C, D y conectivos AND, OR, NOT, XOR, y evaluar la expresión en una entrada concreta.

2. Idea matemática. Una expresión con n variables define una función $f: \{0,1\}^n \rightarrow \{0,1\}$. La tabla de verdad es esa función escrita como tabla: sus 2^n filas recorren todo el dominio, generado con el producto cartesiano $\{0,1\}^n$ en orden binario ascendente. Cada conectivo es una operación del álgebra de Boole: AND = producto, OR = suma booleana, NOT = complemento y **XOR = suma módulo 2**.

En vez de usar `eval()` o `sympy`, se implementó un intérprete propio en tres etapas: **tokenizador** → **parser descendente recursivo** → **evaluador del árbol de sintaxis**. La gramática codifica la precedencia NOT > AND > XOR > OR, y se aceptan varias sintaxis equivalentes (&/AND/*, |/OR/+, !/~NOT, ^/XOR) además de los símbolos del enunciado ($\wedge \vee \neg \oplus$).

Para la documentación — ¿cómo se relaciona una tabla de verdad con un circuito lógico? Son dos descripciones de lo mismo:

Álgebra de Boole	Circuito digital
Variable A	cable/entrada
Conectivo AND, OR, NOT, XOR	compuerta
Expresión completa	red de compuertas
Una fila de la tabla	un estado eléctrico de las entradas
Columna de salida	lo que mide un voltímetro en la salida
Subexpresión repetida	salida reutilizada (fan-out)

La tabla de verdad es la **especificación** («qué debe hacer el circuito») y la expresión es una **implementación** («con qué compuertas»). Como la tabla tiene 2^n filas y cada una puede valer 0 o 1, hay 2^{2^n} funciones distintas de n variables (256 con 3 variables, 65536 con 4), y **cada una es realizable** con AND, OR y NOT: basta escribirla como suma de sus minterminos. Ese es el puente directo con el punto 8. Recorrer la tabla equivale a simular el circuito con todas las entradas posibles, que es precisamente lo que hace un banco de pruebas en electrónica digital.

Las tres expresiones del enunciado $((A \wedge B) \vee (\neg C))$, $(A \oplus B) \wedge C$, $(A \vee B) \wedge (\neg A \vee C)$ se imprimen completas, más una de cuatro variables. Ejemplo:

Expresión: $(A \wedge B) \& C$

A B C | salida

```
-----
0 0 0 0 | 0
1 0 0 1 | 0
2 0 1 0 | 0
3 0 1 1 | 1
4 1 0 0 | 0
5 1 0 1 | 1
6 1 1 0 | 0
7 1 1 1 | 0
```

Filas en 1: 2 de 8 -> minterminos [3, 5]

3. Ejecución. `python3 src/boole/tablas_verdad.py` (o `python3 run_all.py 7`). Para evaluar una entrada concreta: `evaluar("(A & B) | (!C)", {"A":1,"B":1,"C":1})`.

4. Pruebas. Tamaño 2^n de la tabla; vectores de salida exactos de los cuatro conectivos básicos y de las tres expresiones del taller; siete evaluaciones puntuales; equivalencia entre las sintaxis alternativas y los símbolos $\wedge \vee \neg \oplus$; precedencia de operadores (incluyendo un caso que **no** debe ser equivalente); diez leyes del álgebra de Boole (De Morgan, distributivas, absorción, complemento, doble negación); extracción de minterminos; y seis expresiones mal formadas que deben ser rechazadas.

5. Limitaciones. El costo es inevitablemente $O(2^n)$: con muchas variables la tabla es inviable (para eso existen los BDD o los SAT solvers). No hay implicación ($\rightarrow$) ni equivalencia ($\leftrightarrow$) como conectivos primitivos, aunque se pueden escribir con los disponibles ($A \rightarrow B \equiv !A \vee B$). No se simplifican expresiones aquí (eso es el punto 8) ni se dibuja el circuito.

Punto 8 · Simplificación booleana (Quine-McCluskey)

1. Problema. Recibir una función booleana de 3 o 4 variables dada por sus minterminos y producir una expresión simplificada en **suma de productos**, comprobando que conserva la tabla de verdad.

2. Idea matemática. Se implementó **Quine-McCluskey** completo en dos fases:

Fase 1 — implicantes primos. Se aplica repetidamente una sola regla del álgebra de Boole, la **adyacencia lógica**:

$$X \cdot Y + X \cdot \neg Y = X \cdot (Y + \neg Y) = X$$

Dos términos que difieren en exactamente **un** bit se combinan en uno con un guion en esa posición (la variable desaparece). Se repite por rondas; lo que no se puede combinar más es un **implicante primo**.

Fase 2 — cubrimiento mínimo. Elegir el menor número de implicantes primos que cubran todos los minterminos (un problema de cubrimiento de conjuntos). Primero se toman los **primos esenciales** (los únicos que cubren algún mintermino) y luego, para lo que falte, se hace búsqueda exhaustiva por tamaño creciente, lo que da el cubrimiento **exacto** en funciones de este tamaño.

Para la documentación — ¿qué es un mintermino? Es un producto (AND) que contiene **todas** las variables, cada una negada o sin negar, y que por lo tanto vale 1 en **exactamente una** fila de la tabla de verdad. Con A, B, C, el mintermino número 5 = 101 es $A \cdot \neg B \cdot C$. Como cada fila con salida 1 se puede capturar con su mintermino, toda función es la suma (OR) de sus minterminos: esa es la forma canónica suma de productos. En el código un mintermino se representa como cadena de bits ("101") y un implicante como cadena con guiones ("--1" significa «C = 1, A y B da igual»).

Para la documentación — ¿por qué dos expresiones con la misma tabla de verdad son equivalentes? Porque **la tabla es la función**. Una expresión booleana es solo una notación para una función $f: \{0,1\}^n \rightarrow \{0,1\}$, y dos funciones con el mismo dominio son iguales cuando coinciden en todos los puntos del dominio — que es exactamente lo que dice «tener la misma tabla de verdad» (el dominio es finito con 2^n puntos, así que la comparación es completa, no una muestra). La sintaxis puede ser muy distinta y el circuito mucho más barato, pero el comportamiento observable es idéntico. Por eso el programa **verifica** cada simplificación comparando el vector de salida de la forma canónica con el de la expresión simplificada, usando el evaluador del punto 7:

```
Minterminos      : [1, 3, 5, 7]
Forma canonica    : (!A & !B & C) | (!A & B & C) | (A & !B & C) | (A & B & C)
Implicantes primos: ['--1'] -> ['C']
SIMPLIFICADA      : C
```

```
Literales      : 12 -> 1
Misma tabla de verdad que la original: True
```

Este es el **caso sugerido en el taller** ({1,3,5,7} con 3 variables) y el resultado es C, como se esperaba: los cuatro minterminos son 001, 011, 101, 111; A y B recorren todas las combinaciones y solo C es constante igual a 1.

Otro ejemplo con 4 variables, donde se ve el ahorro de compuertas:

```
Minterminos      : [0, 1, 2, 5, 6, 7, 8, 9, 10, 14]
SIMPLIFICADA      : (C & !D) | (!B & !C) | (!A & B & D)
Literales         : 40 -> 7
```

3. Ejecución. python3 src/boole/simplificacion.py (o python3 run_all.py 8). Para otra función: simplificar({1,3,5,7}, ["A","B","C"]).

4. Pruebas. Las piezas por separado (a_cubo, combinar, expansión de un cubo a sus minterminos); el caso sugerido {1,3,5,7} → C; implicantes primos conocidos; conservación de la tabla en siete funciones de 3 y 4 variables; constantes 0 y 1; que la simplificada nunca use más literales que la canónica; **verificación exhaustiva de las 256 funciones de 3 variables**; una muestra de funciones de 4 variables; el cubrimiento mínimo; y minterminos fuera de rango.

Sobre las «herramientas de verificación»: el enunciado pide indicar qué salió del programa propio y qué de la verificación. Aquí **ambas cosas son propias**: la simplificación la produce el Quine-McCluskey de este módulo y la verificación la hace el evaluador de tablas de verdad del punto 7, que es un intérprete independiente escrito para el punto anterior. No se usó sympy (no está instalado en el entorno de desarrollo) ni ninguna otra librería; el control de calidad se apoya en que dos implementaciones distintas coinciden y en la verificación exhaustiva de las 256 funciones de 3 variables.

5. Limitaciones. Quine-McCluskey es exponencial en el peor caso; la búsqueda exhaustiva de la fase 2 es cómoda para 3-4 variables pero no escala (con muchas variables se usaría el método de Petrick o heurísticas como Espresso). Solo produce **suma de productos** (no producto de sumas ni factorizaciones multinivel), no admite condiciones «no importa» (*don't cares*), y minimiza literales, que es una aproximación al costo real de un circuito (no cuenta retardos, fan-in máximo ni compuertas compartidas).

Punto 9 · Entropía de Shannon

1. Problema. Medir cuánta información (incertidumbre) hay en un texto: frecuencias, probabilidades, entropía, y comparar dos mensajes. Como extensión opcional, construir un código de Huffman y comparar su longitud promedio con la entropía.

2. Idea matemática. Si un símbolo s aparece con probabilidad $p(s)$, su «sorpresa» es $-\log_2 p(s)$ bits: un símbolo casi seguro sorprende poco, uno raro sorprende mucho. La entropía es el **promedio** de esa sorpresa:

$$H = - \sum_i p_i \cdot \log_2(p_i) \quad [\text{bits por símbolo}]$$

Propiedades que el programa verifica numéricamente:

- $H = 0$ si hay un solo símbolo (no hay incertidumbre: ya se sabe qué viene).
- H es máxima e igual a $\log_2 k$ cuando los k símbolos son equiprobables.
- $0 \leq H \leq \log_2 k$, y la diferencia $\log_2 k - H$ es la **redundancia**.

Para la documentación — ¿por qué la entropía mide incertidumbre y no longitud? Porque en la fórmula solo entran las **proporciones** p_i , nunca el número de caracteres. Duplicar un texto no cambia ninguna probabilidad, así que no cambia H :

'AB'	longitud 2 -> H = 1.0000
'AABB'	longitud 4 -> H = 1.0000
'AB' repetido 50 veces	longitud 100 -> H = 1.0000

La lectura correcta es que **H son bits por símbolo**: mide cuánto cuesta, en promedio, adivinar el siguiente carácter. La longitud del texto solo aparece si se quiere la información *total* del mensaje, que es $H \times \text{longitud}$. Un texto de mil letras iguales es larguísimo y aun así casi no informa: siempre se sabe qué sigue. Los dos mensajes exigidos por el enunciado:

repetitivo ('AAA...AB', 30 caracteres)	: H = 0.2108 bits/símbolo
variado ('La entropía mide ...')	: H = 3.9931 bits/símbolo

El variado tiene **mayor** entropía porque reparte sus apariciones entre más símbolos y de forma más parecida entre sí, así que predecir el siguiente carácter es más difícil. El repetitivo es casi siempre A: cada símbolo nuevo aporta poca información. Nótese que el repetitivo tiene una redundancia de 0.7892 bits mientras un texto equiprobable tiene redundancia 0 — es decir, el repetitivo se puede comprimir mucho y el equiprobable no.

Extensión: Huffman. Se construye con un algoritmo voraz sobre un montículo: se unen repetidamente los dos nodos de menor frecuencia, de modo que los símbolos raros quedan más profundos (códigos más largos). El teorema de Shannon dice que ningún código libre de prefijos puede tener longitud promedio menor que H , y Huffman cumple $H \leq L < H + 1$. Verificado en cada ejemplo:

```
Huffman para 'variado':
H = 3.9931   L = 4.0185 bits/símbolo   se cumple H <= L < H+1: True
mensaje codificado: 217 bits vs 270 bits con código de longitud fija
decodificación exacta: True
```

```
Huffman para 'AAAABBCD' (p = 1/2, 1/4, 1/8, 1/8):
H = 1.7500   L = 1.7500 bits/símbolo
```

El segundo caso muestra el óptimo perfecto: cuando todas las probabilidades son potencias de $1/2$, $L = H$ exactamente, porque cada $-\log_2 p_i$ es entero y se puede asignar un código de esa longitud exacta.

3. Ejecución. `python3 src/info/shannon.py` (o `python3 run_all.py 9`).

4. Pruebas. Frecuencias y probabilidades (que suman 1); siete valores de entropía conocidos (AAAA $\rightarrow 0$, AB $\rightarrow 1$, ABCD $\rightarrow 2$, AAAABBCD $\rightarrow 1.75$, ...); texto vacío; $H = 0$ solo con un símbolo; $H = \log_2 k$ con k equiprobables para $k = 1..8$; $H \leq \log_2 k$ en 30 textos aleatorios; independencia de la longitud; que el repetitivo tenga menos entropía que el variado; que Huffman sea libre de prefijos; ida y vuelta codificar/decodificar en cinco textos; la cota $H \leq L < H+1$; el caso $L = H$; el borde de un solo símbolo; y bits incompletos al decodificar.

5. Limitaciones. Es la entropía de **orden 0**: se supone que los símbolos son independientes y se ignora toda la estructura del lenguaje. El español real tiene mucho menos de 4 bits por letra de entropía condicional (tras q casi siempre viene u), así que este número **sobreestima** la incertidumbre real. Las probabilidades se estiman con las frecuencias del propio texto, lo que sesga los textos cortos. Huffman se implementó por símbolo y sin serializar la tabla de códigos (que en un compresor real también ocupa espacio).

Punto 10 · Primer simulador cuántico

1. Problema. Simular un qubit: representar su estado, aplicar las compuertas X, Z, H, calcular las probabilidades de medir 0 y 1, y simular 1000 mediciones.

2. Idea matemática. El estado de un qubit es un vector de dos amplitudes complejas

$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, con $|\alpha|^2 + |\beta|^2 = 1$ (normalización)

Las compuertas son matrices 2x2 **unitarias** y aplicarlas es multiplicar matriz por vector (implementado a mano, sin numpy):

X = [[0,1],[1,0]] intercambia $|0\rangle$ y $|1\rangle$ (el NOT cuántico)
 Z = [[1,0],[0,-1]] cambia el signo de la amplitud de $|1\rangle$ (fase)
 H = (1/√2)·[[1,1],[1,-1]] crea superposición

La medición sigue la **regla de Born**: $P(0) = |\alpha|^2$, $P(1) = |\beta|^2$. Ser unitarias es justo lo que garantiza que las probabilidades sigan sumando 1 tras cualquier compuerta (las pruebas verifican $M \cdot M^\dagger = I$ para las tres). Medir además **colapsa** el estado a $|0\rangle$ o $|1\rangle$, así que para repetir el experimento 1000 veces hay que **volver a preparar** el estado antes de cada medida.

Casos obligatorios (salida literal):

```
X|0>          estado = (+0.0000)|0> + (+1.0000)|1>
              1000 medidas -> 0: 0 (0.000), 1: 1000 (1.000)
H|0>          estado = (+0.7071)|0> + (+0.7071)|1>
              P(0) = 0.5000 P(1) = 0.5000
              1000 medidas -> 0: 519 (0.519), 1: 481 (0.481)
HH|0>         estado = (+1.0000)|0> + (+0.0000)|1>
              P(0) = 1.0000 P(1) = 0.0000
Vuelve a |0>: True   (H es su propia inversa: H*H = I)
```

Un detalle interesante que muestra el programa: Z aplicada tras H **no cambia las probabilidades** (0.5, 0.5 en ambos casos) porque la fase relativa no es observable en una medida directa; pero si después se aplica otra H, la diferencia aparece con toda claridad: $HH|0\rangle = |0\rangle$ mientras $HZH|0\rangle = |1\rangle$. Esa es la interferencia, el recurso que usan los algoritmos cuánticos.

Para la documentación — diferencia entre esta probabilidad simulada y un computador cuántico real.

Aquí (simulación)	Computador cuántico real
Las amplitudes se guardan y se pueden leer : se imprime α y β . La probabilidad se calcula con $ \alpha ^2$ y luego se sortea con un generador pseudoaleatorio clásico (random), determinista dada la semilla. No hay ruido: $HH 0\rangle$ devuelve $ 0\rangle$ con error $\sim 10^{-16}$ (solo punto flotante).	El estado es físicamente inaccesible; solo se obtienen resultados de medidas. La aleatoriedad es intrínseca a la mecánica cuántica, no reproducible con una semilla.
Se puede repetir el mismo estado exacto 1000 veces «sin costo». Cuesta $O(2^n)$ memoria: 1 qubit = 2 amplitudes, 50 qubits $\approx 10^{15}$ amplitudes, imposible. El colapso se programa a mano (se reemplaza el vector).	Hay decoherencia, ruido y errores de compuerta y de lectura; $HH 0\rangle$ da $ 0\rangle$ con fidelidad menor a 1 y el resultado se degrada con la profundidad del circuito. Cada repetición (<i>shot</i>) es una ejecución física nueva: preparar, operar y medir. El hardware mantiene el estado de n qubits «gratis»; ahí está la ventaja cuántica potencial. El colapso es un fenómeno físico.

En resumen: la simulación reproduce **la matemática** (vectores, matrices unitarias, regla de Born) con total precisión y de forma inspeccionable, pero no la **física** (ruido, aleatoriedad genuina) ni la **ventaja de escala** (el costo exponencial en memoria es lo que hace útil al hardware cuántico).

3. Ejecución. `python3 src/cuantica/qubit.py` (o `python3 run_all.py 10`). Se pueden encadenar compuertas: `cero().aplicar_circuito("HZH")`.

4. Pruebas. Los tres casos obligatorios ($X|0\rangle = |1\rangle$, $H|0\rangle \approx 50/50$, $HH|0\rangle = |0\rangle$); que X , Z , H sean su propia inversa; que Z no cambie las probabilidades; que HZH actúe como X ; que las probabilidades sumen 1 tras 40 circuitos aleatorios; 1000 mediciones de $H|0\rangle$ dentro de $\pm 5\%$ de 0.5; medidas deterministas para $|0\rangle$ y $|1\rangle$; un estado sesgado ($P(0) = 0.25$) reproducido en 2000 medidas; que la fase compleja no altere las probabilidades; el colapso (medir dos veces da lo mismo); normalización automática de Qubit(3,4); unitariedad $M \cdot M^\dagger = I$ de las tres compuertas; y validaciones de error.

5. Limitaciones. Es **un solo qubit**: no hay entrelazamiento, ni compuertas de dos qubits (CNOT), ni por lo tanto los fenómenos que dan poder a la computación cuántica (extenderlo requiere productos tensoriales y vectores de 2^n entradas). No hay modelo de ruido, ni corrección de errores, ni rotaciones parametrizadas (R_x , R_y , R_z , S , T), ni medición en otras bases. El azar es pseudoaleatorio y las comparaciones usan tolerancias porque los números de punto flotante acumulan error (0.9999999999999996 en vez de 1).

Cierre

Los diez puntos comparten la misma idea de fondo que menciona el enunciado: **estructuras discretas + reglas precisas + procedimientos ejecutables**. Y varias piezas se reutilizan entre puntos, lo que muestra que los temas no son independientes:

- La **aritmética modular** aparece en César ($\mathbb{Z}_{26}$), en RSA ($\mathbb{Z}_n$, $\mathbb{Z}_{\phi(n)}$), en el MPC ($\mathbb{Z}_M$) y en el XOR booleano (suma en $\mathbb{Z}_2$).
- El **algoritmo de Euclides** del punto 2 es lo que hace existir la llave privada.
- **Dijkstra** (punto 4) es el motor del análisis de cierre (punto 5).
- El **evaluador de tablas de verdad** (punto 7) es el verificador independiente de la simplificación (punto 8).
- Los **algoritmos voraces** aparecen en el coloreo (punto 6) y en Huffman (punto 9), y en ambos casos hay que distinguir «solución válida» de «solución óptima»: el voraz del coloreo no garantiza el mínimo, mientras el de Huffman sí alcanza el óptimo entre los códigos libres de prefijos.
- La **enumeración exhaustiva de 2^n casos** es el método de verificación en los puntos 7, 8 y 1 (fuerza bruta), y es también la razón por la que el punto 10 no escala más allá de unos pocos qubits.